

PDFer — Architecture Patch

April 16, 2026 · Recommendations for pre-build revision

> This document is a delta against `pdfer-complete-architecture-summary-post-forge-run-april-15-2026.md`. It captures six changes to apply before the next Forge run (5-instance skeleton build). Rationale included for each so Claude Code can evaluate, push back, or extend before committing.

Change 1 — Surgical LLM: Grok API → Gemma 4 E4B (local)

What changes:

- Replace all references to `Grok 4-1-fast-reasoning API` with `Gemma 4 E4B (local, via llama.cpp)` throughout the architecture doc.
- Update the stack table row.
- Update the tiered installation block.

New tiered install:

```
...
pip install pdfer          # Core: PyMuPDF,
pdfplumber, YOLOv8, FastAPI
pip install pdfer[ocr]    # + Tesseract
pip install pdfer[ocr-pro] # + PaddleOCR
pip install pdfer[llm]    # + llama.cpp + Gemma
4 E4B gguf (default local)
pip install pdfer[llm-api] # + Anthropic/Gemini
API clients (escape hatch for no-GPU users)
...
```

Rationale:

The Grok switch broke the compounding-intelligence story that is PDFer’s core product. Every surgical call sent to a closed API is training data leaving the machine — the opposite of what small-business customers need. Gemma 4 E4B is purpose-built for this:

- Native document/PDF parsing, OCR, chart comprehension, pointing
- Tool calling and structured output native (not bolted on)
- Thinking mode as a tunable quality lever for hard pages

- Apache 2 license, ships in gguf/mlx, runs alongside YOLOv8m on the 4060 Ti

Pre-build verification required:

Before writing extraction prompts, benchmark Gemma 4 E4B on 3 hard pages from a reference corpus (pages with nested tables, multi-column text-wrap, small fonts). Validate that Q4_K_M quantization preserves vision quality for document work — fall back to Q5_K_M if OCR-style tasks degrade. Record latency per page at chosen quant.

Change 2 — Dual Training Loop

What changes:

Replace the single “Correction → Retraining Loop” section with two parallel loops.

Loop A — YOLOv8 (layout detection):

- Trains on corrected bounding boxes from `object\_edits`
- Data: `(page\_image, [corrected\_bboxes\_with\_labels])`
- Existing 5 overfitting safeguards apply
- Versioned: base + last 3

Loop B — Gemma 4 E4B LoRA (extraction quality):

- Trains on corrected extraction outputs from `extractions` table where `object\_edits` shows user modified the content
- Data: `(page\_image\_crop, object\_label, corrected\_markdown\_output)` triples
- LoRA rank 16, alpha 32 as starting point (tune after first run)
- Adapters are small (~50MB) — version last 5
- Auto-reject if held-out BLEU/ROUGE drops >5%

Rationale:

Two compounding loops means the system gets smarter on *your* documents in two independent dimensions: *what* to extract (boxes) and *how* to extract it (outputs). This is the flywheel that separates PDFer from every other PDF tool on the market. “Gets smarter the more you use it, nothing leaves your machine” is the small-business pitch.

Storage impact:

- Training data: stored alongside `object\_edits` (already exists)
- LoRA checkpoints: ~250MB for 5 versions, negligible
- Base Gemma 4 E4B: ~3-4GB gguf depending on quant (one-time)

Change 3 — Staged Retraining Threshold

What changes:

Replace the fixed 200-page retraining trigger with a staged schedule.

New schedule:

[Trigger # Pages corrected Rationale]		
----- ----- -----		
1	50	Base model is dumbest here; early corrections carry the most signal
2	100	Validate the first improvement held
3	200	Standard cadence begins
4	400	
5+	Every 200	Steady state

All triggers configurable. Manual “Train Now” button remains. Per-loop configuration (YOLO and Gemma LoRA can have different schedules).

Rationale:

Diminishing returns curve. A 4B-parameter VLM + a layout detector both benefit most from the first examples of your document style. Waiting 200 pages to first retrain throws away compounding signal you’ve already paid the correction labor for.

Change 4 — Text Spans Feed Stage 1 Post-Processing

What changes:

Update Stage 1 transition contract.

Current text:

```
> 1->2: Stage 1 reads images only (not
text_spans)

**New text:**

> 1->2: Stage 1 reads page images for YOLO
inference. Post-processing (column detection,
reading order, heading hierarchy) uses
text_spans as ground-truth geometry hints.
```

Specific integrations:

- **Column detection:** DBSCAN on text_span X-midpoints (not YOLO object centroids) — text spans are the actual text geometry, far more reliable signal
- **Reading order:** When text_spans exist inside a detected object’s bbox, use their natural order as the intra-object reading order
- **Heading hierarchy:** Font size/bold/caps signatures come from text_spans, not from re-analyzing rasterized images

Rationale:

Text spans are free signal — already extracted in Stage 0. Using them only for extraction in Stage 3 leaves the best geometric hints on the table during the stage where they’d help most.

Edge case handled:

On `scanned-no-ocr` pages, there are no text_spans. Post-processing falls back to YOLO-centroid-only heuristics. This is the degraded-but-working path.

Change 5 — Structured Extraction Failure Format

What changes:

Replace the vague “Failed extractions produce placeholder rows” with a specific schema.

Placeholder row schema in `extractions` table:

```
'''json
{
  "object_id": "...",
  "content_type": "placeholder",
  "content": null,
  "metadata": {
    "failure_type": "llm_timeout | parse_error |
low_confidence | extractor_exception",
    "failed_extractor": "pdfplumber | tesseraect |
gemma_e4b | ...",
    "attempted_at": "2026-04-16T...",
    "retry_count": 0
  },
  "error": "<specific error message>"
}
'''
```

**Placeholder rendering in assembled `document.md`:

...

```
[EXTRACTION_FAILED type=table page=34
bbox=(120,400,450,180) reason=llm_timeout
review=pdfer://doc/abc123/object/xyz789]
...
```

Rationale:

The current spec’s “never a gap” principle is correct. But a gap-filler that reads `[EXTRACTION FAILED]` is useless to a downstream reader. This format is:

- **Auditable:** You can grep for all failures across your corpus
- **Actionable:** The `review=pdfer://...` URI scheme opens the review UI to that exact object
- **Diagnosable:** The metadata tells you whether to retry, fix the extractor, or accept manual-only

Change 6 — Gemma 4 Resolution Verification (New Watch Item)

What changes:

Add to the “Watch Items for Build Run” section.

New watch item:

> **Gemma 4 vision resolution handling.**
Gemma 3 used a fixed 896×896 SigLIP encoder with pan-and-scan for non-square images. Gemma 4 advertises variable aspect ratio and resolution support but the specifics for 1275×1650 page rasters (letter at 150 DPI) aren’t documented in consumer-facing docs yet. If pan-and-scan fires on every page, you’re doing 2-4 encoder passes per page and latency multiplies. Test with: (a) full-page rasters, (b) half-page crops based on YOLO bounding boxes, (c) object-crop-only (just the detected region + small padding). Find the sweet spot between context and latency. Likely winner: object-crop-only for most labels, full-page only for `table` and `figure` with caption proximity.

Rationale:

This is the kind of thing that will eat 30% of the build run’s time if we don’t flag it upfront. Better to have an instance benchmark it in hour one than discover it in hour eight.

Explicitly Dropped From Plan

Squire/Mandrel integration surface:

PDFer is a standalone tool. Mandrel is build infrastructure for PDFer itself (context + project management for the build run), not a runtime component. Squire is a personal assistant, also not a runtime component. PDFer’s output contract is `document.md` + assets/ + metadata.json` in a directory. What downstream tools do with that output is out of scope for v1.

Ready-to-Build Checklist (Pre-Forge)

Before starting the 5-instance skeleton build run, the following must exist:

1. **Test harness skeleton:** pytest configured, reference PDF in `tests/fixtures/`, smoke test that imports the (not-yet-written) pipeline module and asserts it exists
1. **Mandrel project initialized** with PDFer build context
1. **Seed document** defining: TODO list structure, test-fix loop rules, “refuse to build on broken foundation” clause, instance state reporting format
1. **Gemma 4 E4B gguf downloaded** and smoke-tested via llama.cpp on one page
1. **Benchmark data collected:** 3 hard pages from reference corpus processed through Gemma 4 E4B at Q4_K_M, latency and output quality recorded

Open Questions for Claude Code Review

1. **Gemma LoRA training infrastructure:** llama.cpp doesn’t natively train LoRA. Do we use `unsloth` or `peft` for the fine-tuning step and merge back to gguf for inference? This affects the `pip install pdfer[llm]` dependency weight.
1. **Is there value in ensemble extraction for the first 50 corrections?** Run pdfplumber *and* Gemma 4 on the same object, let the human pick, capture both attempts as training signal. Double cost for first 50, then switch to single-extractor after first LoRA trigger.
1. **YOLOv8 and Gemma 4 sharing the GPU:** Detection is bursty (one shot per document), LLM surgical calls are bursty (~10 per document). Do they contend in practice, or does the pipeline naturally sequence them? Confirm or add an explicit GPU scheduler.
1. **Should the “Train Now” button require a named dataset?** e.g., “Train on all NWFA corrections” vs “Train on everything.” Lets users build domain-specific checkpoints they can swap.

Ready to hand to Claude Code 4.7 for review and patching into the main architecture doc.